Lab1
Pneumonia Classification

TA: 良亦 & 致雅

Important Rule

Submission Deadline: 2026/10/20 (Tue.) 23：59

Late submission: grade * 80%

Turn in:

1）Experiment Report (.pdf) to E3 「LAB1_yourstudentID_name.pdf」eg : 「LAB1_314553018_王小致.pdf」

2）Source Code (.py) to your own github

(a)「train.py」「inference.py」.....

(b) README.md

Lab Objective

Pneumonia Classification in chest X-ray images

Object 1: write your own custom DataLoader

Object 2: Pneumonia Classification (binary classification: Normal / Pneumonia**)**

Object 3: Evaluation

Requirements

1. Implement **at least 2 ResNet architectures** (ResNet18, ResNet50, ResNet101 … or others), calling model from pytorch API is allowed.

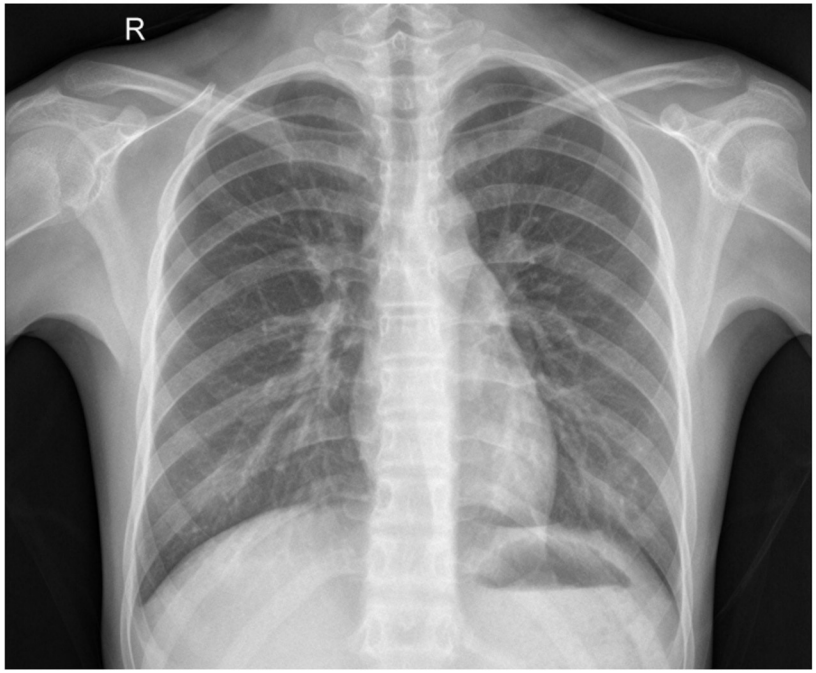
2. **Visualize and compare the accuracy and F1-score trends of different models.** Plot the accuracy and F1-score for each training epoch (required). Validation curves are optional and worth a 5-point bonus. **Plot the confusion matrix** of the final test results.

3. **Upload you code to your own github** including README.md

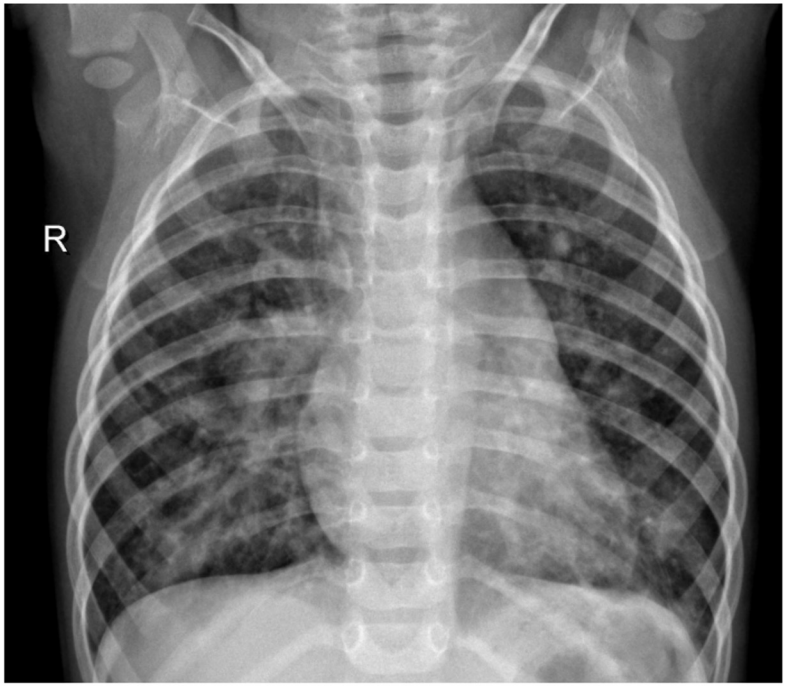
Dataset - Chest X-ray (kaggle)

https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia

NORMAL

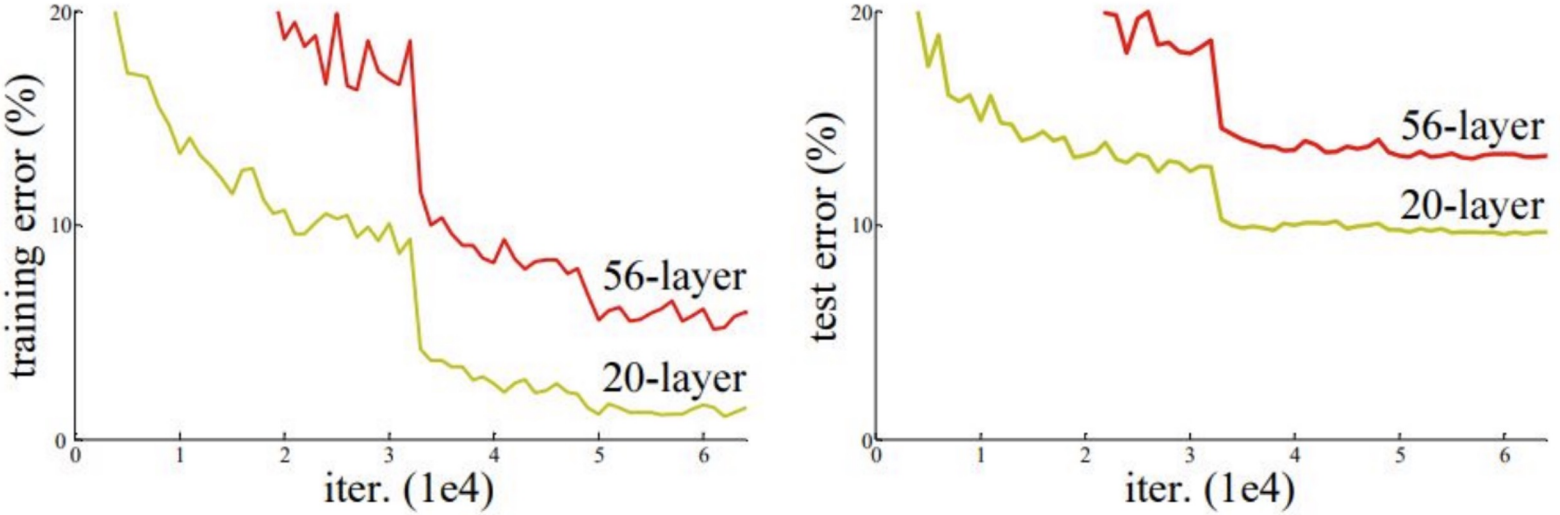


Pneumonia



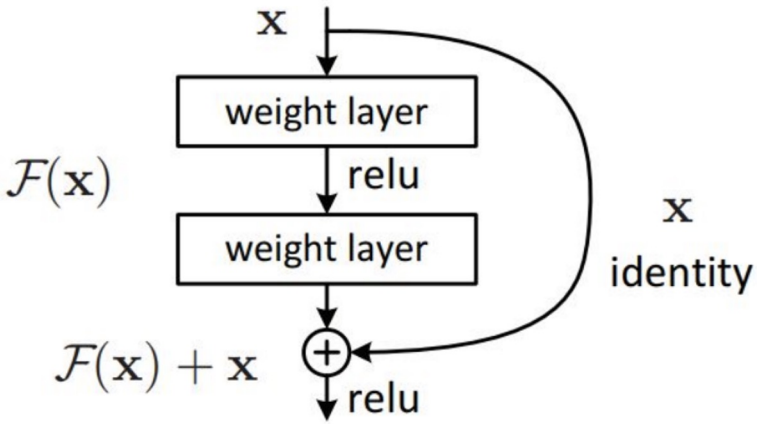
ResNet

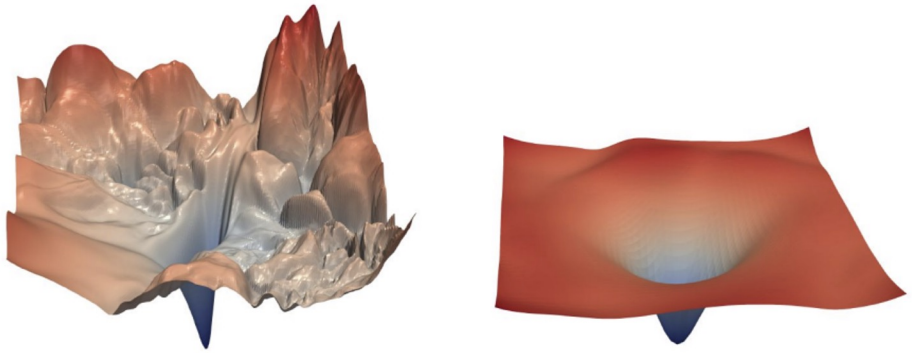
ResNet (Residual Network) is the Winner of ILSVRC 2015 in image classification, detection, and localization, as well as Winner of MS COCO 2015 detection, and segmentation



ResNet

To solve the problem of vanishing/exploding gradients, a skip / shortcut connection is added to add the input x to the output after few weight layers as below





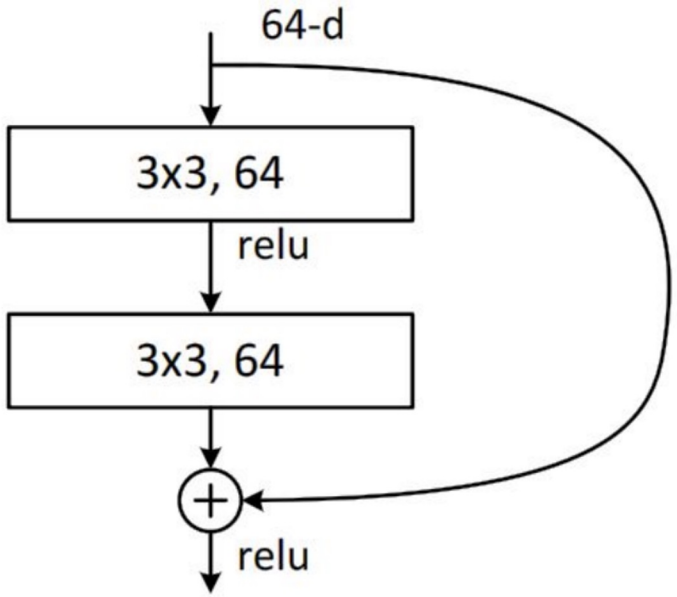
(a) without skip connections (b) with skip connections

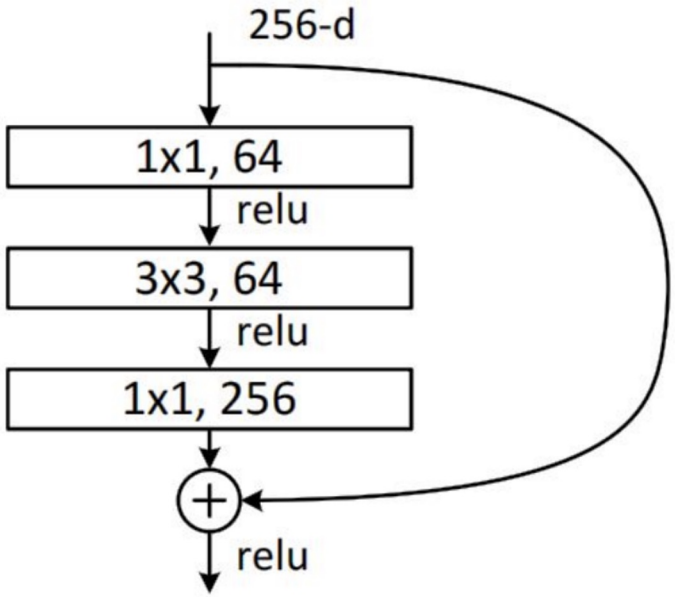
Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

Source: Li, Hao, et al. "Visualizing the loss landscape of neural nets." Advances in Neural Information Processing Systems. 2018.

ResNet

ResNe18(Basic block), ResNet50(Bottleneck block)





Using Pretrained Model

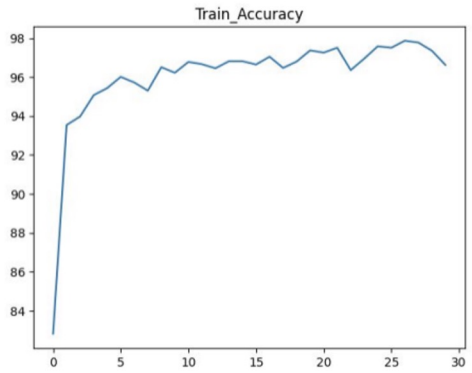
Using pretrained model by torchvision module

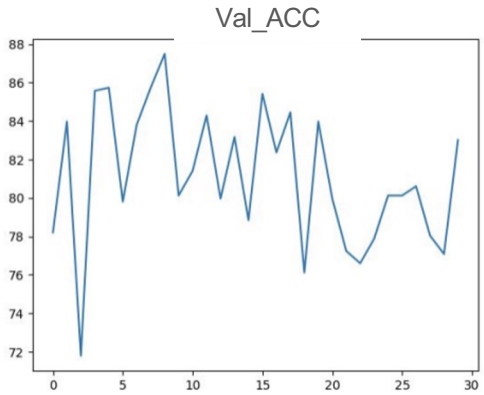
```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
              :
    )
  (layer2): Sequential(
              :
    )
  (layer3): Sequential(
              :
    )
  (layer4): Sequential(
              :
    )
  (avgpool): AvgPool2d(kernel_size=7, stride=1, padding=0)
  (fc): Linear(in_features=512, out_features=1000, bias=True)
)
```

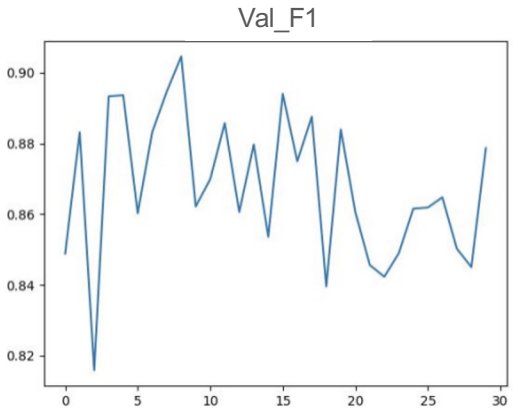
You need to reinitialize the specific layers

Result Comparision

Compare and visualize the accuracy and F1-score trend in training phase (validation phase is optional, **you'll get a 5 point bonus if you visualize the result of validation phase**)

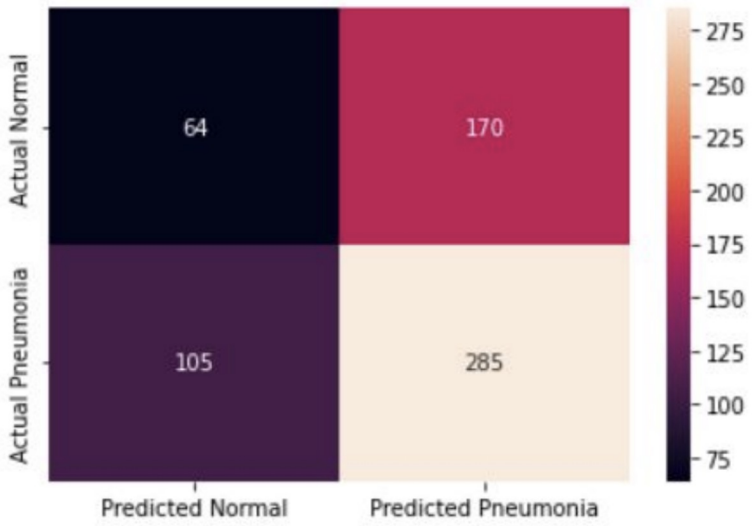






Confusion Matrix

Calculate the confusion matrix and plotting



How to plot on Python

Tutorial:

https://matplotlib.org/stable/tutorials/introductory/pyplot.html
https://seaborn.pydata.org/generated/seaborn.heatmap.html

Report Spec

1. Introduction (5%)
2. Experiment setups (25%)
    a. The detail of your model
    b. The detail of you Dataloder (e.g. different data augmentation methods)
3. Experiment result (30%)
    a. Highest testing accuracy and F1-score (screenshot)
    b. Plotting the comparison figure
        i. Training accuracy and F1-score curves
        ii. Validation accuracy and F1-score curves (optional, 5-point bonus)
        iii. Confusion matrix of the final test results
    c. Anything you want to present
4. Discussion (35%) (**Most important part**)
    a. Discuss your discovery or share anything you want
5. Github Link (5%) (Do not forget)

Score criterion of Lab1

Score: 30% experimental results + 70% report

P.S **If the report exists format errors (file name or the report spec), it will be 5 points penalty (-5)**

---- Criterion of result (30%) ----

Accuracy > = 90% = 100 pts

Accuracy 80~90% = 90 pts

Accuracy 70~80% = 80 pts

Accuracy < 70% = 70 pts

Notice

1. You may use PyTorch and torchvision models.
2. ImageNet pretrained weights are allowed. Please reinitialize the final classification layer.
3. Make sure others can reproduce your results using your code and README.md.

Reference

[1] He, Kaiming, et al. "Deep residual learning for image recognition." Proceedings of the IEEE conference on computer vision and pattern recognition. 2016.

[2] https://www.kaggle.com/paultimothymooney/chest-xray-pneumonia

Download the dataset after registration